

Hassan Lindsay
CS499 Computer Science Capstone
4-2 Milestone Three: Enhancement Two, Algorithms and Data Structures
May 29, 2026

Enhancement Two: Algorithms and Data Structures

For this milestone, I enhanced the Fire Department Event Tracker application by improving how event data is searched, filtered, organized, and displayed within the application. The purpose of this enhancement was to demonstrate knowledge and practical application of algorithms and data structures through real-world software improvements that improve usability, efficiency, and maintainability.

The original artifact allowed users to create and store department events such as drills, meetings, training sessions, and parades using SQLite database storage and RecyclerView components. While the application successfully displayed event information, the original implementation lacked efficient searching and chronological organization of event data. Users had to manually scroll through the full event list to locate specific events, and events were not automatically sorted by upcoming date and time.

To improve the functionality of the artifact, I implemented a dynamic event search filtering system. The search enhancement allows users to filter the RecyclerView event list in real time while typing into the search field. The filtering logic checks event titles, dates, times, and locations to determine matching records. This enhancement was implemented using an ArrayList data structure combined with linear traversal logic.

The search process uses linear search, which operates with $O(n)$ time complexity because each event in the list must be checked individually during the filtering process. Although linear search is less efficient than binary search for very large datasets, it was an appropriate choice for this mobile application. The expected event volume is relatively small, and the filtering logic

must dynamically evaluate multiple event fields, including title, date, time and location. For datasets such as department event schedules containing dozens or even hundreds of records, the performance impact remains minimal on modern mobile devices. All this while still maintaining simple and maintainable code.

In addition to search filtering, I implemented chronological event sorting so that upcoming events are automatically organized by date and time. This enhancement improves usability by ensuring users see the next scheduled events first rather than viewing unsorted event data. The sorting process uses comparator-based sorting with parsed Date objects generated from the stored event date and time strings. Converting the stored string values into Date objects allowed the application to perform accurate chronological comparisons, since direct string comparison would not have been very reliable to sort dates and times in proper calendar order.

The chronological sorting enhancement operates with approximately $O(n \log n)$ time complexity because it relies on Java's built-in sorting mechanisms for collections. Helper methods were created to convert date and time strings into comparable Date objects, allowing events to be sorted accurately while improving maintainability and readability of the codebase.

Additional improvements included updating the RecyclerView dynamically whenever filtering or sorting occurs, improving code organization through reusable helper methods, and strengthening overall maintainability of the application. These enhancements improved both the user experience and the internal structure of the software artifact.

One challenge encountered during development involved properly parsing event dates and times while maintaining compatibility with the existing SQLite database structure. Another challenge involved ensuring that invalid dates and improperly formatted time values did not

interfere with sorting behavior. These issues were resolved through improved validation logic and reusable parsing helper methods.

Overall, this enhancement strengthened the application by improving event organization, user interaction, and software maintainability while demonstrating practical implementation of algorithms, data structures, traversal logic, time complexity analysis, and real-world software engineering practices. While completing these enhancements, I also realized that a future improvement for the application would be implementing automatic event archiving or deletion once events have passed. As event records continue to accumulate over time. This added enhancement could further improve long term data organization and application efficiency. I will explore this feature post-graduation as I continue to expand my software development skills.